Technical Requirements Document (TRD)

1. Architecture & Components

The application is built using the **MERN Stack** (MongoDB, Express, React, Node.js), utilizing a decoupled client-server architecture.

- **Frontend (Client):** A Single Page Application (SPA) built with React. It uses Context API for state management (Auth & Cart) and React Router for navigation.
- **Backend (API):** A RESTful API built with Express.js.
- **Security Layer:** Custom middlewares handle authentication via JSON Web Tokens (JWT). The `protect.js` middleware extracts the token from the `Authorization: Bearer` header. The `admin.js` middleware ensures route access is restricted to users with the `admin` role.
- **Error Handling:** A centralized error handler (`error.js`) manages duplicate entries (code 11000), validation errors, and invalid JWT tokens globally.

2. Infrastructure Requirements

- **Database:** MongoDB, managed via Mongoose Object Data Modeling (ODM).
- **Server:** Node.js runtime environment.
- **Environment Variables:** Managed via `dotenv` for securing secrets like the `JWT_SECRET` and `API_URL`.

3. Development Tools, Frameworks, and Libraries

Frontend (Client-Side)

- **React (v18.2.0):** Core UI library.
- **React Router Dom (v7.15.0):** For routing and protected navigation.
- **Formik (v2.4.9):** Used for form state management and validation in the Checkout process.
- **Bootstrap (v5.3.8):** For responsive UI styling and layout grids.

Backend (Server-Side)

- **Express (v5.2.1):** Web framework for handling HTTP requests and routing.
- **Mongoose (v9.5.0):** MongoDB object modeling tool designed to work in an asynchronous environment.
- **Bcrypt (v6.0.0):** For hashing user passwords before database storage.
- **JSON Web Token (v9.0.3):** For generating secure access tokens.
- **Cors (v2.8.6):** Cross-Origin Resource Sharing middleware to allow frontend-backend communication.
- **Morgan (v1.10.1):** HTTP request logger middleware.

4. System Integrations (API Endpoints)

The RESTful API is structured into several modular routes:

- **Auth:** `/register` and `/login`.
- **Users:** `/profile` (Protected) for normal users, and `/` or `/:id` (Protected, Admin) for administration.
- **Products:** Public access to view products, while creating, updating, or deleting requires Admin privileges.
- **Cart:** Integrates directly with the `Cart` model, linking a unique User ID to an array of product items.
- **Orders:** Generates an order containing `user`, `items`, `totalPrice`, `status` (default: pending), and `shippingAddress`. The frontend checkout form sends a POST request with the shipping payload.
- **Reviews:** Allows logged-in users to post ratings (1-5) and comments, linked to specific Product IDs.
- **Categories:** Retrieves category definitions (name, description).

5. Database Architecture

